Vintage of 4 Seasons - site + Stripe checkout + order emails (Cloudflare Pages)

Cloudflare Pages' free plan allows commercial use, so it can host a real shop.

What is in this folder

- public/index.html the website
- public/og-image.jpg the picture shown when the site link is shared
- public/robots.txt and public/sitemap.xml for Google
- functions/api/checkout.js the secure Stripe checkout (holds the real prices)
- functions/api/stripe-webhook.js emails each paid order to the shop's Gmail
- test/checkout.test.mjs and test/webhook.test.mjs optional self-checks (node test/xxx.test.mjs)

Part 1 - Put the site and checkout online

1. GitHub: new repository, upload the folders in this package (keep "functions" and "public" as they are).
2. Cloudflare dashboard (dash.cloudflare.com) > Workers & Pages > Create > Pages > Connect to Git.
   Pick the repository. Build settings:
   - Framework preset: None
   - Build command: exit 0 (Cloudflare recommends this so the checkout functions are picked up)
   - Build output directory: public
     Click Save and Deploy. (Cloudflare changes button names now and then - tell Claude what you see.)
3. In the project: Settings > Variables and secrets > add
   - STRIPE_SECRET_KEY = your Stripe key (use the test key, sk_test_..., first) - set the type to Secret
   - SITE_URL = your real address, starting with https:// (for now, the *.pages.dev address Cloudflare gives you)

Then redeploy (Deployments > the latest one > Retry deployment).

4. Domain: if you bought it at Cloudflare, open the project > Custom domains > Set up a custom domain.
   Then change SITE_URL to the domain and redeploy.
5. Replace YOURDOMAIN.com in public/index.html, public/sitemap.xml and public/robots.txt
   (or tell Claude the domain and ask for a fresh package).
6. Test order: card 4242 4242 4242 4242, any future date, any CVC. Check it appears in Stripe.
   Then switch STRIPE_SECRET_KEY to the live key (sk_live_...), redeploy, and place one small real order.

Part 2 - Email every paid order to the shop's Gmail (optional)

Card orders live in Stripe. This part also emails the details (items, total, customer, shipping address)
to the Gmail every time a payment goes through.

1. Make a free account at resend.com. SIGN UP WITH THE SAME GMAIL THAT SHOULD RECEIVE THE ORDERS.
   (Until a domain is verified, Resend only delivers to the address you signed up with.)
2. In Resend: API Keys > Create API key. Copy it (starts with re_).
3. In Cloudflare > your project > Settings > Variables and secrets, add:
   - RESEND_API_KEY = the Resend key (Secret)
   - ORDER_EMAIL = the Gmail address that should get the orders
4. In Stripe (Dashboard) > Developers > Webhooks > Add endpoint:
   - Endpoint URL: https://YOUR-SITE-ADDRESS/api/stripe-webhook
   - Event to listen for: checkout.session.completed
     After saving, click the endpoint and reveal its Signing secret (starts with whsec_).
5. In Cloudflare add STRIPE_WEBHOOK_SECRET = that whsec_ value (Secret). Redeploy.
6. Test in Stripe test mode: place a test order, then check the Gmail (and its spam folder).
   In Stripe's webhook page you can see every delivery and whether it worked.
7. When you switch to live mode: Stripe's live mode has its OWN webhook endpoint and signing secret.
   Add the endpoint again in live mode and update STRIPE_WEBHOOK_SECRET along with the live STRIPE_SECRET_KEY.
8. Later, when the domain is bought and verified in Resend, add ORDER_FROM, for

example:
Vintage of 4 Seasons orders@yourdomain.com so the emails come from your own address.

Free limits: Resend allows 100 emails a day and 3,000 a month. Cloudflare's free plan allows 100,000
function requests a day. Both are plenty for a small shop.

Keep in mind

- Prices live in functions/api/checkout.js AND on the website. Change both.
- Never put any key in the website or in GitHub. Only in Cloudflare's Variables and secrets.
- If an order email is missing, check Stripe: Developers > Webhooks > your endpoint > recent deliveries.